

Reading One Line

The seven questions to ask of any TypeScript line, in order, with worked examples and the note you write before moving on
October 2026

The method.

For every line, ask the same seven questions in the same order, then write one note saying what the label on the left now holds, as a concrete value. The next line's questions read that note. Under a minute per line once it is a habit; the whole review is this pass, line by line, feeding the sixteen review questions.

The seven questions

Q1. What kind of line is it?

Exactly one of:

Kind	Looks like	What to expect
declaration	<code>const</code> / <code>let</code> / <code>function</code> / <code>class</code> / <code>interface</code> / <code>type</code>	a new label (or type) comes into existence
assignment	<code>x = ...</code> , <code>x += ...</code> , <code>o.k = ...</code> , <code>arr[i] = ...</code>	an existing label is re-pointed, or an object is mutated
expression statement	a bare call: <code>arr.push(1)</code> ; <code>save()</code> ; <code>name.trim()</code> ;	something runs; the result goes nowhere unless the call mutates
control	<code>if</code> / <code>for</code> / <code>while</code> / <code>return</code> / <code>throw</code> / <code>try</code> / <code>await</code> alone	changes which line runs next, or ends the function
type-only	after <code>:</code> , inside <code>interface</code> / <code>type</code> , <code>as T</code> , <code>x!</code>	runs nothing; skip Q3–Q7; note only what the author is claiming

Q2. Where does the value come from? Find the right-hand side (after `=`, or the whole expression if there is no `=`). Name its producer:

Producer	Examples
literal	<code>{}</code> , <code>[]</code> , <code>{ a: 1 }</code> , <code>"a"</code> , <code>5</code> , <code>true</code> , <code>null</code>
call	<code>f(x)</code> , <code>obj.method()</code> , <code>new C()</code> , <code>fetch(url)</code>
property read	<code>o.k</code> , <code>o["k"]</code> , <code>arr[i]</code> , <code>arr.length</code> , <code>map.get(k)</code>
operator	<code>a + b</code> , <code>a ?? b</code> , <code>a b</code> , <code>c ? a : b</code> , <code>!x</code> , <code>...a</code>
<code>await</code>	<code>await p</code> : the producer is whatever <code>p</code> resolves to
destructuring	<code>const { a } = o</code> : the producer is <code>o.a</code>

Q3. Primitive or object? The value from Q2 is one of the seven primitives (`string`, `number`, `boolean`, `undefined`, `null`, `bigint`, `symbol`) or an object (array, plain object, `Map`, `Set`, `Date`, `Promise`, function, class instance). Primitive: immutable, compared by value, copied; Q6 does not apply. Object: mutable, compared by identity, aliased; Q6 applies.

Q4. Ticket or value?

Only if Q2 was a call.

- Does the call return a `Promise`? Yes if the function is declared `async`, annotated `: Promise<...>`, is `fetch` / `res.json` / `db.query` / `new Promise` / `.then` / anything from `fs/promises` or a client SDK, or its body `return`s one of those.
- If yes: is there an `await` on this line, or is the result `returned`, `.then`'d, or passed to `Promise.all`?
- If no to step 2: the label holds the ticket, not the value. The next `.property` on it is `undefined`. Q7 answer is “not kept”; this is the missing-await finding.

Q5. Can it be nothing? Can the value from Q2 be `undefined` or `null`?

Source of nothing	Why
a property marked <code>?</code>	may be absent
<code>arr.find(...)</code> , <code>arr.findLast(...)</code>	no match
<code>map.get(k)</code>	key absent
<code>arr[i]</code>	past the end, or a hole
<code>req.headers[...]</code> , <code>req.query.x</code> , <code>req.params.x</code> , <code>req.body.x</code>	not sent
a function with a path that falls off the end	no <code>return</code> on that path
<code>?.</code> anywhere in the expression	short-circuits to <code>undefined</code>

Source of nothing	Why
a database row lookup	not found, often <code>null</code>
<code>JSON.parse</code> of external data	any shape at all
a default parameter when the caller passed <code>null</code>	default applies only to <code>undefined</code>

If yes: does the next use of this label check (`if (x)`, `x == null`, `x ?? d`, `x?.k`) or walk straight into `.something`? Also note whether the check uses `!x`, which wrongly treats `0`, `"` and `false` as nothing.

Q6. Who else points here?

Only if Q3 was an object.

1. Did this line make a **new address**, or an **alias** to an existing one?

New address	Alias
a literal <code>{}</code> , <code>[]</code> , <code>new Map()</code>	<code>= a</code> (another label)
<code>[...a]</code> , <code>{ ...o }</code> , <code>a.slice()</code> , <code>Array.from(a)</code>	a parameter (points at the caller's object)
<code>a.map(f)</code> , <code>a.filter(p)</code> , <code>a.concat(b)</code> , <code>toSorted</code> , <code>toReversed</code>	<code>find</code> , <code>arr[i]</code> , <code>map.get(k)</code> (the actual element, not a copy)
<code>structuredClone(o)</code>	<code>return this.items</code> (internal state handed out)
<code>JSON.parse(JSON.stringify(o))</code>	a cache hit
	a closure capturing an outer object

2. If alias: does this line, or any later line, **mutate** it? (`push`, `pop`, `shift`, `unshift`, `splice`, `sort`, `reverse`, `fill`, `o.k = v`, `delete o.k`, `arr.length = 0`, `map.set`, `set.add`, `date.setX`.)
3. If alias and mutated: who else holds that address, and do they expect it unchanged? That is the finding.
4. Copies are shallow unless `structuredClone: { ...o }` shares `o.inner`; `filter` shares the elements.

Q7. Was the result kept? Does the line's value go somewhere: assigned to a label, `returned`, `await`ed, passed as an argument, used in a condition, or stored into an object or collection?

Bare expression statement	Kept?	Verdict
<code>arr.push(x);</code> , <code>arr.sort();</code> , <code>o.k = v;</code> , <code>map.set(k, v);</code>	the mutation IS the effect	fine
<code>arr.map(f);</code> , <code>arr.filter(p);</code> , <code>arr.slice();</code>	new array dropped	dead line
<code>s.trim();</code> , <code>s.toUpperCase();</code> , <code>s.replace(...)</code>	new string dropped	dead line
<code>fetch(url);</code> , <code>save(x);</code> (async, no <code>await</code>)	Promise dropped	fire-and-forget; rejection unhandled
<code>arr.map(async ...)</code> with no <code>Promise.all</code>	array of Promises dropped	same
<code>f(x);</code> where <code>f</code> is sync and has side effects	the side effect IS the effect	fine
<code>x;</code> , <code>a + b;</code> , <code>o.k;</code>	nothing	dead line

The note

After Q7, write one line:

Label holds: , , , .

Examples:

- `user` holds: a `User` object shared with `users[]`, or `undefined`
- `ids` holds: a new `number[]` sorted as strings
- `p` holds: `Promise<User>`, a ticket, not `await`ed
- `size` holds: a number, possibly `NaN`, possibly `0`
- `opts` holds: a new object; `opts.inner` is shared with `defaults.inner`

The next line's Q5 and Q6 are answered by reading this note, not by re-deriving.

Worked lines

Line A

```
const user = users.find((u) => u.id === req.params.id);
```

Q	Answer
1	declaration with assignment
2	a call: <code>users.find(...)</code> with an arrow condition
3	object (a <code>User</code>) or <code>undefined</code>
4	<code>find</code> is synchronous; not a ticket
5	yes: <code>find</code> returns <code>undefined</code> when nothing matches; and <code>req.params.id</code> is a string, so if <code>u.id</code> is a number, <code>===</code> never matches and this is always <code>undefined</code>
6	alias: <code>find</code> returns the actual element; mutating <code>user.x</code> later mutates the element inside <code>users</code>
7	kept in <code>user</code>
note	<code>user</code> holds: a <code>User</code> object shared with <code>users[]</code> , or <code>undefined</code>

Next line `res.json({ name: user.name })`: Q5 from the note says `user` can be `undefined`; `.name` on it crashes for an unknown id. Finding.

Line B

```
const ids = orders.map(o => o.id).sort();
```

Q	Answer
1	declaration
2	a chain: <code>map</code> then <code>sort</code>
3	object (array of numbers)
4	synchronous; no ticket
5	an array, never <code>undefined</code> ; can be empty
6	<code>map</code> made a new array; <code>sort</code> mutates that new one, not <code>orders</code> ; safe
7	kept in <code>ids</code>
note	<code>ids</code> holds: a new number[] sorted as strings (no comparator): <code>[1, 10, 9]</code>

Finding if numeric order was required: `.sort((a, b) => a - b)`.

Line C

```
const data = res.json();
```

Q	Answer
1	declaration
2	a call: <code>res.json()</code>
3	object: a Promise
4	<code>res.json()</code> returns a Promise; there is no <code>await</code> ; <code>data</code> holds the ticket
5	not <code>undefined</code> , but not the body either
6	new Promise object; not shared
7	kept, but as a ticket
note	<code>data</code> holds: <code>Promise<any></code> , a ticket, not awaited

Next line `data.user`: `undefined`. Finding: missing `await`.

Line D

```
const opts = { ...defaults, ...input };
```

Q	Answer
1	declaration
2	a literal with two spreads
3	object

Q	Answer
4	no call; not a ticket
5	not <code>undefined</code> ; individual keys may be
6	new address for <code>opts</code> ; shallow: any nested object in <code>defaults</code> or <code>input</code> is shared; later keys win, so <code>input</code> overrides <code>defaults</code> (correct order)
7	kept
note	<code>opts</code> holds: a new object, input over defaults; <code>opts.nested</code> shared

Line E

```
if (order.status = "paid") total += order.amount;
```

Q	Answer
1	control, containing an assignment
2	the condition is an assignment expression; its value is <code>"paid"</code>
3	primitive string
4	no call
5	<code>"paid"</code> is truthy, always
6	the assignment mutates <code>order</code> , which is an element of the caller's array
7	the condition's value is used; it is always true
note	<code>order.status</code> is now <code>"paid"</code> for every order; branch always runs

Finding: `=` for `===`; corrupts input and counts every order.

Line F

```
cache.set(key, await load(key));
```

Q	Answer
1	expression statement
2	a call <code>cache.set</code> whose second argument is <code>await load(key)</code>
3	<code>set</code> returns the Map (object); the awaited value is whatever <code>load</code> resolves to
4	<code>load</code> is awaited here; <code>set</code> is synchronous
5	<code>key</code> could be <code>undefined</code> if it came from a header; then every keyless caller shares one entry
6	<code>cache</code> is shared module state; the <code>await</code> is a yield point: between the <code>has</code> check on the previous line and this <code>set</code> , another request can run the same miss
7	the mutation is the effect
note	<code>cache</code> now holds <code>key -> value</code> ; race on concurrent misses; grows if keys are unbounded

“Released or not”

No line releases anything directly. A value stays alive while any label, array slot, Map entry, closure or timer points at it, and is reclaimed when the last one lets go. So the release question is Q6 run once at the end of each function, not per line:

1. Which labels go out of scope when the function returns? Their values are released, unless something else still points at them.
2. Which values were stored into something that outlives the function: a module-level `Map` or array, a cache, a listener, a `setInterval` closure, a returned object? Those are not released.
3. For each of those: who removes it later, and when? If nobody, it grows forever.

The seven in one line each

1. **Kind:** declaration, assignment, expression, control, or type-only?
2. **Source:** literal, call, property read, operator, `await`, or destructuring?
3. **Primitive or object?**
4. **Ticket or value?** (calls only) Promise-returning, and awaited?

5. **Can it be nothing?** `undefined/null` possible, and checked before the next `.`?
6. **Who else points here?** (objects only) new address or alias; mutated; who else holds it?
7. **Was the result kept?** assigned, returned, awaited, passed, used, or dropped?

Then the note: `label` holds: *value, shared/own, nothing-possible, ticket/value*.